

基于Apache Arrow的TPC-H Q5 CPU-GPU协同查询实验

徐子桓 2025104082

摘要 本文实现了一个固定的TPC-H Q5内存查询执行器，用它比较CPU列式算子、专用CPU执行、三种CUDA内存模式、cuDF算子库和CPU-GPU混合执行。六张输入表先统一转换为Apache Arrow IPC，CPU端建立过滤传播索引，GPU端主要扫描lineitem并按国家聚合。与只测一次冷启动不同，本实验增加了常驻会话，把数据装载和内存准备记为setup，把后续请求单独计时。SF1和SF10各包含18组配置、54次预热和180次正式请求，所有后端均通过oracle。SF1中copy、managed和mapped的请求中位数分别为1.267、1.440和22.971 ms；SF10中分别为15.416、15.066和358.582 ms。固定比例混合执行在两个规模的最佳中位数为1.160和10.054 ms，但自动比例相对离线最佳点仍有33.89%和9.21%的损失。Nsight结果进一步表明，mapped虽然没有显式输入H2D，核函数仍会经PCIe远程读取主机页。实验也暴露出setup成本较高、自动模型不稳定等问题。因此本文的结论只适用于当前Q5实现、测试硬件和两个数据规模，不能直接推广到通用数据库。

关键词 内存数据库；TPC-H Q5；Apache Arrow；CUDA；统一虚拟寻址；CPU-GPU协同

Abstract This report implements a fixed TPC-H Query 5 executor and compares an Arrow CPU plan, a specialized CPU path, three CUDA memory modes, cuDF, and concurrent CPU-GPU execution. Arrow IPC is the common input format. A resident-session protocol separates one-time setup from repeated request latency. For both SF1 and SF10, 18 configurations completed 54 warmups and 180 measured requests with oracle-verified results. At SF10, the resident medians of copy, managed, and mapped memory were 15.416, 15.066, and 358.582 ms. The best fixed hybrid ratio reached 10.054 ms, while the automatic policy had 9.21% regret relative to the offline fixed-ratio sweep. Nsight evidence confirms that mapped memory removes an explicit input copy but still causes remote PCIe reads during the kernel. The prototype remains a fixed-query executor, uses only two scale factors, and has substantial setup costs. Its results should not be generalized to a full DBMS or other hardware without further experiments.

Keywords in-memory database; TPC-H Q5; Apache Arrow; CUDA; UVA; CPU-GPU execution

1 引言

GPU具有较高的并行度和显存带宽，但数据库查询不仅有核函数计算，还包含数据装载、哈希或索引准备、主机与设备之间的数据移动等步骤。如果只报告CUDA核函数时间，很容易忽略真正占用时间的部分。本实验选择TPC-H Q5，是因为它同时包含地区和日期过滤、六表连接、聚合与排序，能够把CPU执行、GPU内存访问和CPU-GPU协同放在同一查询语义下讨论。

项目不是通用数据库。它没有SQL解析器、逻辑重写和事务模块，而是把Q5直接写成物理执行计划。这样做的优点是课程周期内可以把正确性、内存模式和实验流程做完整；缺点是专用优化较多，结论的外推范围比较小。本文主要回答四个问题：（1）不同后端能否得到同一Q5结果；

（2）常驻请求下copy、managed、mapped的差别是什么；（3）CPU与GPU按数据分片后能否获得加速；（4）简单的自动比例模型能否代替离线扫描。最后一个问题的结果并不理想，但我仍把它保留在报告中。

2 查询语义与Arrow数据

Q5一共使用六张表。region和nation确定目标地区中的国家；customer和orders确定客户、订单日期及订单所属国家；supplier给出供应商国家；lineitem保存订单明细和价格。一个lineitem被计入结果，需要订单日期位于1994年、客户与供应商都属于ASIA，并且二者来自同一国家。最后按国家计算

$$\text{revenue} = \sum l.\text{extendedprice} \times (1 - l.\text{discount}). \quad (1)$$

同国家条件不是为了提高性能而人为增加的，它来自Q5本身，用来统计一个地区内由本国供应商完成的订单收入。

项目只保存Q5需要的列。字符串采用字典编码，日期转换成从Unix epoch开始的天数。价格用分表示，折扣用百分之一表示，单行收入在 10^4 尺度上计算为

$$r_{10^4} = p_{cent} \times (100 - d_{0.01}). \quad (2)$$

聚合完成后再舍入到两位小数。这一细节很重要：早期版本逐行截断不足一分的部分，虽然不同后端彼此一致，却和官方答案不一致。现在的正确性门禁同时检查内部结果哈希和官方oracle，避免出现“大家一起算错”的情况。

数据准备脚本把TPC-H的.tbl转换为Arrow IPC文件，并记录schema、行数、record batch和SHA256。C++与cuDF都从这组文件开始，因此输入来源一致。不过Arrow统一的是接口和列式格式，不代表各后端没有额外转换：专用CPU会建立直接索引，CUDA会准备设备可访问数组，cuDF会构造DataFrame。这也是后文把setup和request分开统计的原因。

3 系统实现

3.1 CPU物理计划

专用CPU执行器先找到ASIA中的nation，再构造三个直接索引数组：供应商 $S[s]$ 、客户 $C[c]$ 和订单 $O[o]$ 。不满足地区或日期条件的位置记为-1。扫描lineitem时只需检查 $O[o] = S[s] \neq -1$ ，然后把定点数收入累加到对应国家。线程按Arrow batch边界和连续行区间分工，各自使用本地聚合数组，最后归并。这个方法避免物化六表连接，但依赖TPC-H主键较稠密以及查询固定的条件。

Arrow Acero基线使用filter、hash join、aggregate和order by算子组成较通用的列式计划。它与专用路径语义相同，却不能使用相同程度的直接索引优化。因此两者的性能差异反映“通用算子计划与固定查询专用计划”的综合差异，不能简单解释为Arrow框架本身慢。

3.2 三种CUDA内存模式

三个手写CUDA后端执行同一个lineitem聚合核，只改变输入数组的存放和访问方式：

- gpu-copy在setup时分配device buffer并显式H2D复制；
- gpu-managed使用统一内存，运行时通过预取和页迁移使GPU访问；

- gpu-mapped使用映射的锁页主机内存，GPU通过UVA地址经PCIe读取。

常驻会话中，copy的输入H2D发生在setup，不在每次request里。managed也不是“完全不用传输”，它只是由运行时管理迁移。mapped没有大块输入H2D，但数据仍要跨PCIe，只是传输发生在核函数读取主机页时。

3.3 混合执行与自动比例

混合后端按Arrow batch把lineitem切成互不重叠的CPU和GPU区间。GPU异步处理一段，主线程同时扫描另一段，结束后按国家合并两个定点数结果。固定模式扫描0.125到0.875共7个CPU比例。自动模式在session setup中用抽样估计CPU和GPU单位处理能力，再选择一个比例，正式请求不重新调参。

这个模型故意较简单，只使用行数和一次校准，没有考虑PCIe访问、CPU缓存、不同batch大小和运行时干扰。我原先希望它能接近离线最优解，实际只在SF10比较接近，SF1偏差较大。这是当前实现中一个比较明确的漏洞。

3.4 常驻会话

每个后端都提供setup和重复request两个阶段。setup负责Arrow装载、索引构建、内存分配及必要的数据传输；request只执行已经准备好的查询。若setup时间为 T_s ，请求中位数为 T_q ，本文用

$$T_{amortized}(n) = T_s/n + T_q \quad (3)$$

估计 n 次请求的平均成本。它不是并发吞吐模型，只表示同一会话顺序重复请求时一次性成本如何摊薄。

4 实验方法

首先使用6条lineitem的tiny数据检查边界条件和收入公式。正式数据由TPC-H V3.0.1 dbgen生成，包含SF1与SF10。每个规模有18组配置：专用CPU和Acero各3种线程数，copy、managed、mapped、cuDF各1组，固定混合7个比例，自动混合1组。每组建立一次session，预热3次，再测量10次，因此每个规模共有18组配置、54次预热和180次正式请求。

主指标是request的中位数和p95，同时记录setup以及1、10、100次请求的摊销结果。正式计时不包含profiler采集。正确性要求每个请求状态为ok、哈希唯一，并与冻结oracle逐行一致。SF1和SF10证据manifest的摘要分别为81c48484b397和2a9e07aaaeb8。

实验服务器为双路AMD EPYC 9654，GPU为NVIDIA GeForce RTX 4090，驱

动版本595.71.05；软件包含CUDA、Apache Arrow/PyArrow 23.0.1和cuDF 26.06。正式证据固定到提交021becd1b10f。此外，对SF1/SF10的copy、managed、mapped、0.5固定混合和auto混合各采集一组Nsight Systems与Nsight Compute。profiler会明显扰动执行时间，所以它只用于解释内存活动、NVTX阶段和核函数指标，不进入普通延迟排名。

5 实验结果

5.1 正确性

SF1的180次正式请求得到唯一哈希542abf4003633c7c，SF10的180次请求得到b1351a421ba8dcfd。两个规模的18组配置全部通过oracle。表1给出SF1官方答案；核对过程使用定点整数，没有先转换成浮点数。

Table 1 SF1 Q5结果与官方答案核对

国家	收入
INDONESIA	55502041.17
VIETNAM	55295087.00
CHINA	53724494.26
INDIA	52035512.00
JAPAN	45410175.70

5.2 常驻请求性能

表2列出各后端最有代表性的请求中位数。专用CPU从SF1的3.201 ms增长到SF10的14.955 ms。Acero分别为311.461和3124.388 ms，说明当前通用连接计划的开销仍然较大。cuDF在SF10为27.906 ms，虽然比Acero快很多，但也没有超过专用CUDA和最佳混合路径。

Table 2 常驻请求时间中位数 (ms)

后端	SF1	SF10
专用CPU	3.201	14.955
Arrow Acero	311.461	3124.388
cuDF	12.773	27.906
gpu-copy	1.267	15.416
gpu-managed	1.440	15.066
gpu-mapped	22.971	358.582
hybrid-fixed最佳	1.160	10.054
hybrid-auto	1.553	10.980

三种内存模式的变化并不是一个固定排名。SF1中copy比managed低约0.17 ms；SF10中managed反而比copy低约0.35 ms，这个差距相对p95波动并不大，我

不认为它足以证明managed在更大规模一定更好。mapped在两个规模都明显最慢，SF10达到358.582 ms。它避免的是setup中的显式复制，不是PCIe数据访问。

5.3 混合比例与自动模型

SF1固定混合的最佳CPU比例是0.125，中位数1.160 ms；SF10最佳比例变为0.375，中位数10.054 ms。两个规模都低于对应的专用CPU请求时间，说明当数据已经常驻并且分片合理时，CPU-GPU并行可以带来收益。它与V5冷进程中混合执行较慢并不矛盾，因为V5把重复的数据准备也算在每次运行中。

auto在SF1预测比例0.243，实际选择0.262，请求中位数1.553 ms，相对最佳fixed的regret为33.89%。SF10的预测和选择比例为0.289与0.288，中位数10.980 ms，regret降到9.21%。模型捕捉到了规模变化方向，但还不能替代离线比例扫描。

5.4 setup与摊销

请求快不代表第一次查询也快。SF10最佳fixed的setup为4997.298 ms，因此1次请求的摊销值仍有5007.352 ms，100次请求后才降到60.027 ms。专用CPU的SF10 setup为3496.051 ms，100次摊销值为49.915 ms，仍低于混合路径。这说明当前混合后端虽然稳态请求更快，但需要足够多的重复请求才能抵消准备成本。auto的setup更高，SF10达到7844.079 ms，主要来自两侧计划准备和校准。这里还有明显优化空间。

5.5 Profiler观察

下文把Nsight Systems和Nsight Compute分别简称为NSYS和NCU。

NSYS在SF10测得copy核函数约15.34 ms。

managed约15.59 ms，mapped约135.25 ms。

NCU测得的三个数值分别为16.72、16.93和129.81 ms。

copy与managed的device DRAM读取均约545 MB，mapped只有约2.89 MB。这不是mapped少读了数据，而是主要输入来自映射主机页，没有经过device DRAM。mapped的occupancy反而较高，因此occupancy本身不能代表有效吞吐。

0.5固定混合在SF10的NCU核函数约8.16 ms，DRAM读取约273 MB，与只处理部分lineitem相符。NSYS中CPU scan约12.31 ms、GPU请求约8.27 ms，二者位于约12.51 ms的measured request范围内，支

持CPU和GPU阶段存在时间重叠。不过profiler运行比普通请求慢，本文不把这些数值加入表2，也不据此计算端到端加速比。

6 讨论与不足

实验支持了“专用物理计划比通用Acero计划更适合固定Q5”以及“常驻状态下合理分片可以让CPU和GPU共同受益”这两个判断，但没有支持“copy在所有规模都最好”和“简单auto模型可以直接找到最优比例”。我一开始比较关注GPU核函数，做完常驻实验后才发现setup会改变结论：SF10最佳混合请求只要10.05 ms，但建立会话接近5 s。如果老师只问单次查询，专用CPU仍更合适；如果同一数据上重复很多次，常驻GPU和混合执行才有讨论价值。

本项目还存在以下限制。第一，它只实现固定Q5，CPU直接索引不适合任意SQL。第二，GPU只处理最后的lineitem扫描和聚合，六表连接的大部分准备仍在CPU。第三，Arrow统一了输入文件，却没有统一各后端内部表示，公平性仍有争议。第四，只有SF1、SF10和一台服务器，没有多查询并发、NUMA绑定或其他GPU。第五，自动模型只做一次简单校准，SF1 regret较大。第六，Nsight重放会扰动统一内存行为，分析器数值只能解释机制，不能替代普通实验。最后，实验每组只建立一个session，setup本身没有做多次统计，这会让摊销结果对一次性抖动比较敏感。

这些问题并不表示实验没有意义，但会限制结论。后续更值得做的是复用CPU过滤索引、减少混合两侧重复准备，再增加session级多次setup和并发请求实验。我认为继续微调当前原子聚合核的收

益，可能还不如先解决这些端到端问题。

7 结论

本文完成了一个基于Arrow输入的TPC-H Q5 CPU-GPU实验原型，并把冷启动、常驻请求和profiler观察分开记录。SF1/SF10的全部正式请求通过oracle.copy与managed在常驻请求中接近，mapped因远程PCIe访问明显较慢；固定比例混合在两个规模都超过了专用CPU请求时间，但setup成本较高；auto能够跟随规模改变比例，却没有达到离线最优点。这个结果说明，CPU-GPU查询优化不能只看是否并行或是否使用统一地址，还要同时考虑生命周期、数据放置、比例选择和准备成本。由于系统仍是固定查询执行器，本文把这些结论看作对Q5实现的实验观察，而不是通用数据库结论。

- [1] Transaction Processing Performance Council. TPC Benchmark H Standard Specification Revision 3.0.1[EB/OL]. <https://www.tpc.org/tpch/>.
- [2] Apache Arrow Project. Apache Arrow Documentation[EB/OL]. <https://arrow.apache.org/docs/>.
- [3] NVIDIA. CUDA C++ Programming Guide[EB/OL]. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>.
- [4] RAPIDS. cuDF Documentation[EB/OL]. <https://docs.rapids.ai/api/cudf/stable/>.
- [5] NVIDIA. Nsight Systems and Nsight Compute Documentation[EB/OL]. <https://docs.nvidia.com/nsight-systems/>.